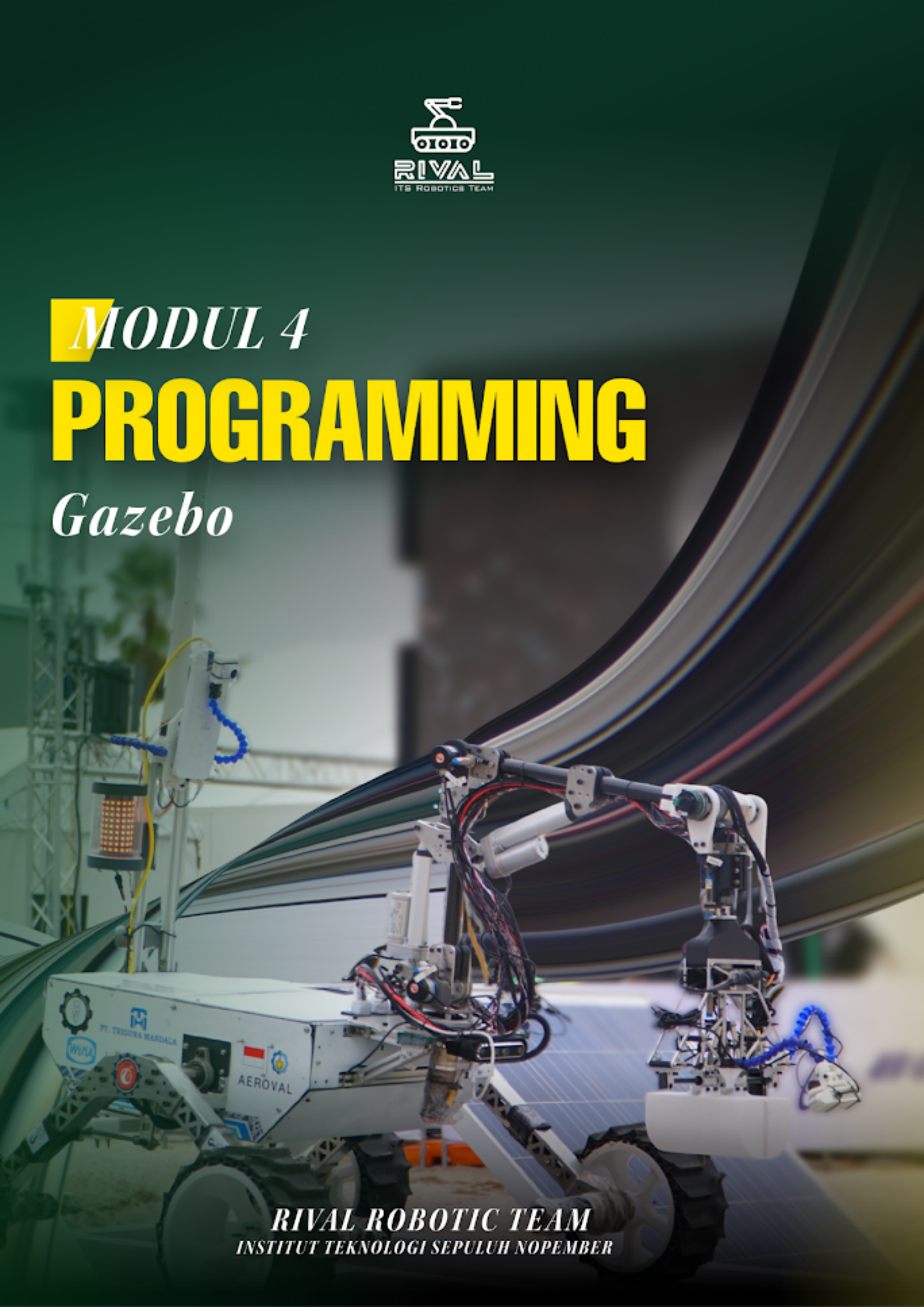
RIVAL
ITS ROBOTICS TEAM

MODUL 4

PROGRAMMING

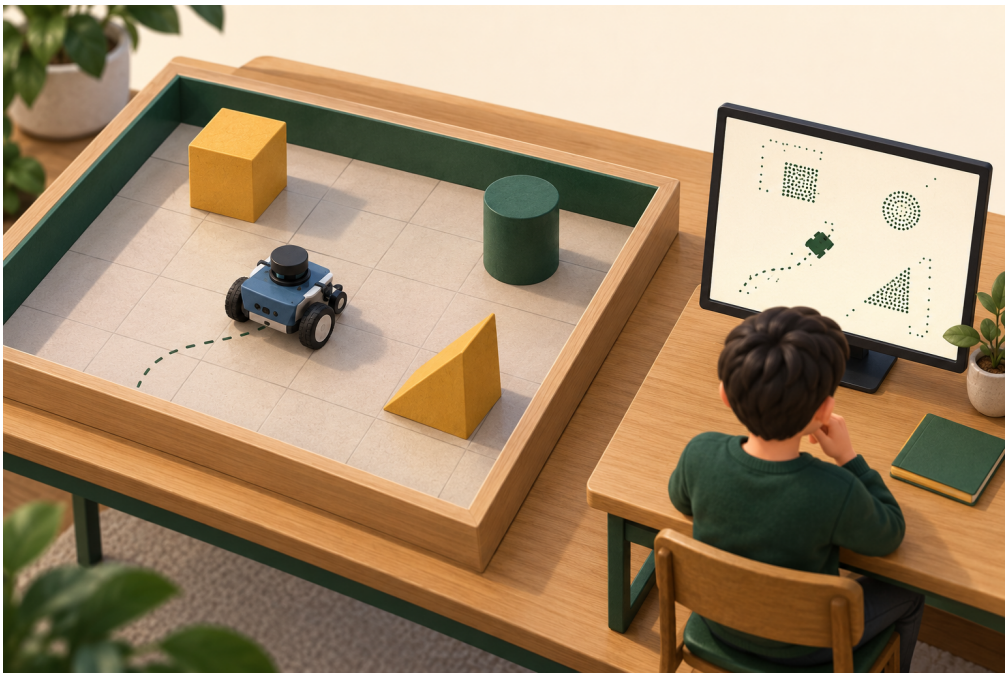
Gazebo



RIVAL ROBOTIC TEAM
INSTITUT TEKNOLOGI SEPULUH NOPEMBER

Gazebo Simulation & RViz2

Gazebo dan RViz2 itu apa?



Gambar 1. Perumpamaan arena latihan dan meja pemantau. Benda di arena dan data pada monitor punya hubungan, tetapi tugas keduanya berbeda.

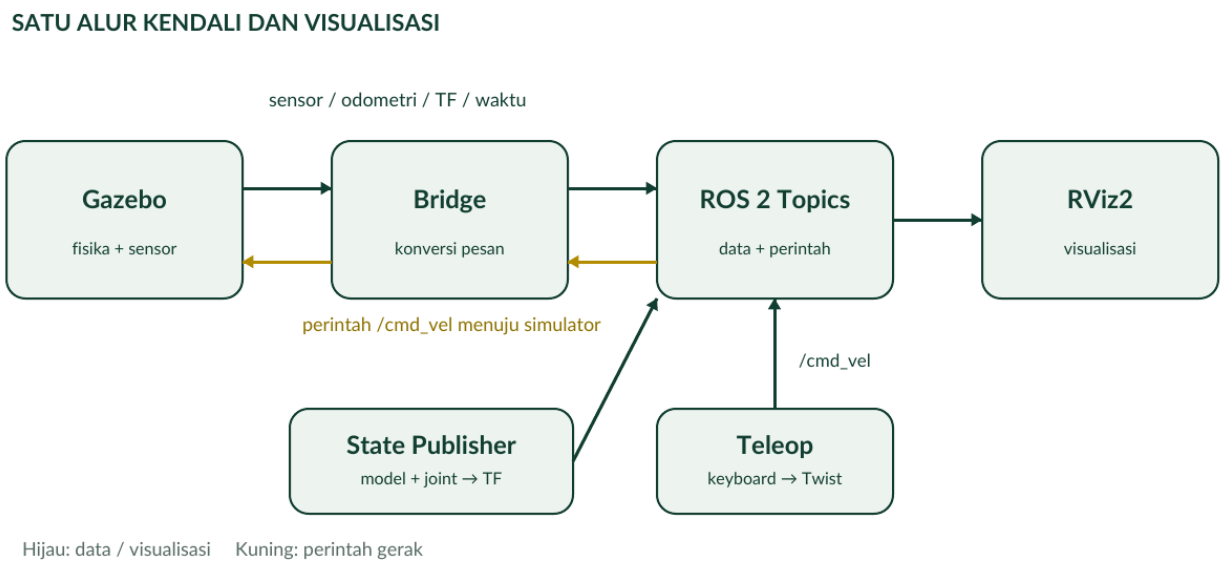
Bayangin kamu punya arena latihan kecil: ada robot, lantai, dan balok penghalang. Gazebo menjalankan arena virtual itu, termasuk gerakan roda, tabrakan, dan pembacaan sensor. Jadi, kita bisa mencoba robot sebelum memakai perangkat aslinya.

RViz2 seperti layar pemantau di sebelah arena. Ia menerima data dari ROS 2 lalu menggambarkannya: bentuk robot, posisi, arah, sampai jarak yang terbaca sensor. Karena itu, Gazebo menampilkan tembok utuh, sedangkan RViz2 bisa hanya menampilkan titik pada permukaan yang terbaca LiDAR.

Di sini kita pakai robot beroda dari template open source Gazebo yang sudah disesuaikan. Kita akan menjalankannya, menggerakkannya dengan keyboard, lalu membuka kode yang membuat semuanya bekerja. Cuplikan kode di modul menjelaskan bagian tertentu; file lengkapnya tetap ada di package.

Gazebo Simulation & RViz2

Alur data robot



Gambar 2. Perintah bergerak menuju simulator. Data sensor, odometri, dan waktu mengalir kembali ke ROS 2 untuk dipakai node lain.

ROS 2 menghubungkan program-program kecil yang disebut node. Misalnya, node teleop membaca keyboard, sementara RViz2 menampilkan data. Mereka bertukar message (pesan dengan struktur tertentu) lewat topic, yaitu saluran bernama seperti /cmd_vel atau /scan.

Publisher adalah pengirim pesan; subscriber adalah penerimanya. Bayangin topic seperti kanal radio: pengirim dan penerima harus memilih kanal dan format pesan yang cocok. Nama topic saja belum menjamin data sampai.

Bagian	Yang dikerjakan
Gazebo	Menjalankan fisika, benda, robot, dan sensor virtual.
ROS 2	Menghubungkan bagian-bagian program melalui pesan dan antarmuka lain.
ros_gz_bridge	Mengubah pesan ROS 2 ke pesan Gazebo, atau sebaliknya.
RViz2	Menggambar data dan model berdasarkan frame yang tersedia.

Gazebo memakai sistem pesan sendiri. ros_gz_bridge menjadi penerjemah antara pesan Gazebo dan ROS 2. Saat tombol ditekan, perintah melewati bridge menuju robot. Data sensor dan posisi bergerak ke arah sebaliknya.

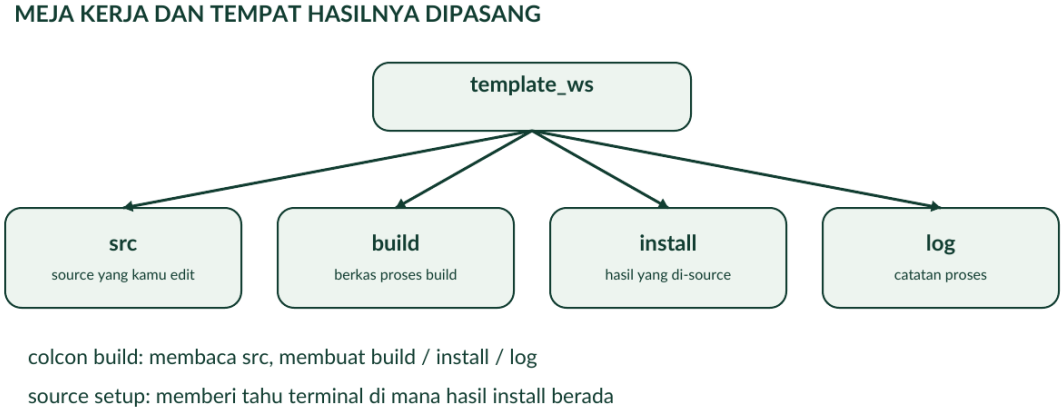
Gazebo Simulation & RViz2

Siapkan package-nya dulu

Ekstrak paket latihan, lalu buka terminal di folder gazebo-rviz2 yang berisi scripts dan template_ws. Gunakan Bash. Ubuntu 22.04 memakai ROS 2 Humble, Ubuntu 24.04 memakai Jazzy; keduanya memakai Gazebo Harmonic dan perintah praktik yang sama.

```
bash
bash scripts/install_dependencies.sh --check
bash scripts/install_dependencies.sh --install
bash scripts/build.sh
```

--check memeriksa dependensi; --install memasangnya lewat apt dan menampilkan perubahan sebelum konfirmasi. ROS 2 dasar perlu sudah terpasang. Pada Humble, bridge Fortress dapat diganti dengan bridge Harmonic. Jika ROS 2 belum terpasang, ikuti panduan instalasi resmi untuk distro yang sesuai.



Gambar 3. Workspace seperti meja kerja. Source adalah bahan yang kamu edit, sedangkan install berisi hasil yang ditemukan ROS 2 saat package dijalankan.

Workspace adalah folder kerja ROS 2; di sini namanya template_ws. Package adalah kelompok file bernama di dalamnya. Edit file di src, bukan build atau install. Build menyiapkan program dan aset ke install; log berisi catatan prosesnya.

Source latihan ada di template_ws/src/ros_gz_project_template. Package ros_gz_example_bringup berisi launch dan konfigurasi, ros_gz_example_description berisi model, sedangkan ros_gz_example_gazebo berisi world.

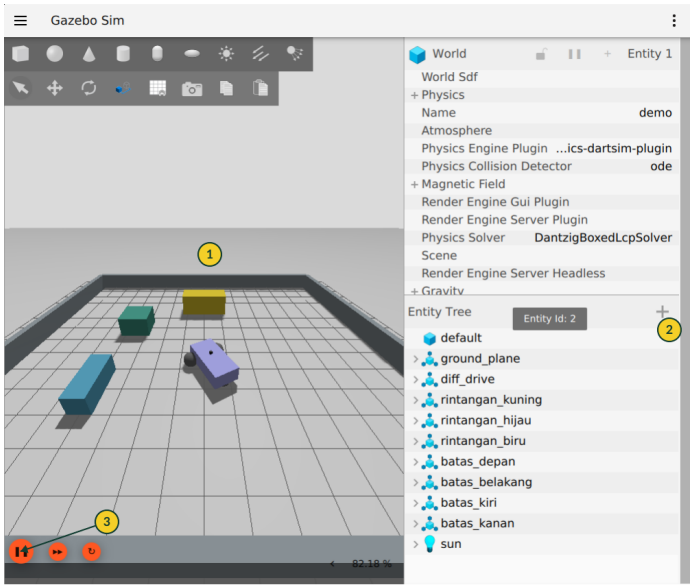
Gazebo Simulation & RViz2

Jalankan dan lihat arenanya

```
source scripts/setup_env.bash
ros2 launch ros_gz_example_bringup diff_drive.launch.py
```

Jalankan dari folder modul. Source memuat pengaturan ROS 2 dan workspace ke terminal saat ini. Biarkan terminal ini terbuka sebagai Terminal A; launch akan membuka Gazebo, bridge, robot_state_publisher, dan RViz2.

Gazebo · world dan robot



1 Viewport: amati robot dan rintangan.
2 Entity Tree: pilih objek world.
3 Play/pause: jeda atau lanjutkan simulasi.

Gambar 4. Gazebo menampilkan robot, rintangan, daftar objek, dan kontrol simulasi. Penanda pada gambar membantu menemukan bagian antarmukanya.

Di Gazebo, world adalah seluruh lingkungan simulasi. Robot dan rintangan di dalamnya disebut model; entity adalah istilah umum untuk objek yang dikelola simulator. Coba geser kamera dan zoom supaya robot terlihat jelas. Jika simulasi pause, tekan Play sebelum mencoba bergerak.

Kalau berpindah terminal, source scripts/setup_env.bash lagi. Setelah selesai praktik, hentikan robot dari teleop, keluar dari teleop dengan Ctrl+C, lalu hentikan launch di Terminal A dengan Ctrl+C.

Gazebo Simulation & RViz2

Menggerakkan robot dari keyboard

Buka Terminal B di folder modul, lalu jalankan node keyboard. Pastikan simulasi tidak sedang pause dan fokus keyboard berada pada terminal ini.

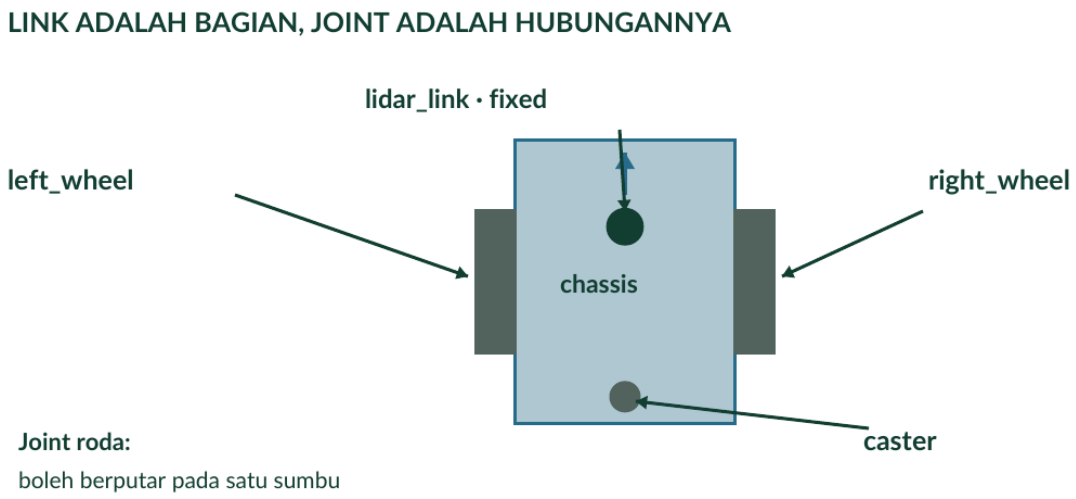
```
source scripts/setup_env.bash
ros2 run teleop_twist_keyboard teleop_twist_keyboard
```

Tombol	Yang terjadi
i / ,	Maju / mundur.
j / l	Berputar kiri / kanan di tempat.
u / o	Maju sambil belok kiri / kanan.
k	Mengirim kecepatan nol untuk berhenti.
q / z	Menaikkan / menurunkan kecepatan linear dan angular pada teleop.
w / x	Mengubah kecepatan linear saja.
e / c	Mengubah kecepatan angular saja.
Ctrl+C	Keluar dari node teleop.

Klik Terminal B, gunakan huruf kecil, lalu coba i untuk maju dan k untuk berhenti. Melepas tombol tidak selalu mengirim perintah berhenti. Tekan k sebelum pindah jendela; z bisa dipakai untuk memperlambat gerakan. Teleop mengirim geometry_msgs/msg/Twist ke /cmd_vel. Twist berisi kecepatan linear dan angular. Untuk robot ini, linear.x mengatur maju-mundur (m/s), sedangkan angular.z mengatur putaran (rad/s). Misalnya x=0,2 dan z=0,4 meminta robot maju sambil belok kiri.

Gazebo Simulation & RViz2

Mengenal bagian model robot



Gambar 5. Link bisa dibayangkan seperti bagian-bagian benda. Joint menentukan cara dua bagian tersebut terhubung atau bergerak.

Link adalah bagian kaku, seperti badan atau roda. Joint adalah hubungan antarlink. Joint roda memungkinkan putaran, sedangkan joint tetap mengikat sensor pada badan. Plugin adalah komponen tambahan yang memberi perilaku, misalnya penggerak roda atau sensor.

Tiga hal yang sering tertukar

Visual menentukan bentuk dan warna yang terlihat. Collision menentukan bentuk yang dipakai saat menghitung tabrakan. Inertial menjelaskan massa dan momen inersia, yaitu sifat benda yang memengaruhi responsnya terhadap gaya dan putaran.

Bayangin robot memakai kostum: visual adalah kostumnya, collision adalah bentuk yang bisa tersenggol, sedangkan inertial menggambarkan berat dan sebaran massanya. Bentuk collision boleh disederhanakan supaya simulasi lebih ringan, asalkan tetap masuk akal.

Pose berarti posisi sekaligus orientasi. Dalam SDF, urutannya x y z roll pitch yaw: tiga angka pertama dalam meter, tiga sisanya dalam radian. Roll, pitch, dan yaw adalah putaran terhadap sumbu x, y, dan z.

SDF dan URDF

File di package	Dipakai untuk
models/diff_drive/model.sdf	Mendeskripsikan model fisika, sensor, dan sistem gerak di Gazebo.
worlds/diff_drive.sdf	Menyusun lingkungan, memasukkan robot, dan mengaktifkan sistem world.
models/diff_drive/visual.urdf	Memberi deskripsi model dan hubungan link untuk RobotModel serta robot_state_publisher.

SDF, singkatan Simulation Description Format, bisa mendeskripsikan lingkungan simulasi sekaligus model di dalamnya. URDF, singkatan Unified Robot Description Format, umum dipakai di ROS untuk mendeskripsikan struktur robot.

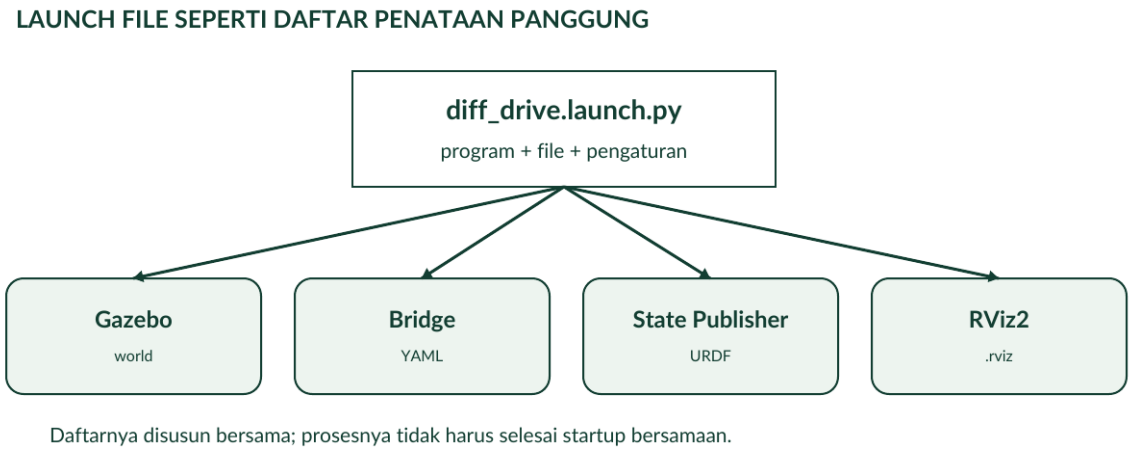
```
<sdf version="1.8">
  <model name="diff_drive">
    <!-- link, joint, sensor, dan plugin -->
  </model>
</sdf>
```

```
<robot name="diff_drive">
  <link name="base_link" />
  <!-- link dan joint untuk model visual -->
</robot>
```

Di package adaptasi ini, visual.urdf dibuat sebagai pendamping model SDF supaya visualisasi memakai jalur yang sama pada Humble dan Jazzy. URDF tersebut bukan model fisika lengkap. Kalau mengubah bentuk atau susunan link di SDF, ingat untuk menyesuaikan model visualnya juga.

Gazebo Simulation & RViz2

Fungsi utama di launch file



Gambar 6. Launch file seperti daftar penataan panggung: program apa yang dibuka, file apa yang dibaca, dan pengaturan apa yang dibawa.

Buka ros_gz_example_bringup/launch/diff_drive.launch.py. Fungsi utamanya bernama generate_launch_description(). ROS 2 launch memanggil fungsi ini untuk memperoleh daftar tindakan yang perlu dijalankan.

```
def generate_launch_description():
    # Menentukan lokasi file dan menyiapkan tiap bagian.
    # Isi lengkapnya ada di file package.
    return LaunchDescription([
        gz_sim,
        DeclareLaunchArgument('rviz', default_value='true',
                              description='Open RViz.'),
        bridge,
        robot_state_publisher,
        rviz
    ])
```

generate_launch_description() mengembalikan daftar tindakan lewat LaunchDescription. gz_sim, bridge, robot_state_publisher, dan rviz adalah variabel yang disiapkan sebelumnya. ROS 2 launch membaca daftar ini lalu menjalankan tindakannya.

Node(...) mendeskripsikan program ROS 2, sedangkan IncludeLaunchDescription(...) memasukkan launch lain. Menulis objeknya belum langsung membuka proses. Program juga belum tentu siap bersamaan, jadi beri waktu sampai simulator dan datanya muncul.

Gazebo Simulation & RViz2

Blok pembuka Gazebo

```
gz_sim = IncludeLaunchDescription(
    PythonLaunchDescriptionSource(
        os.path.join(pkg_ros_gz_sim, 'launch',
                     'gz_sim.launch.py')),
    launch_arguments={
        'gz_args': ['-r ', PathJoinSubstitution([
            pkg_project_gazebo,
            'worlds',
            'diff_drive.sdf'
        ])]
    }.items(),
)
```

IncludeLaunchDescription memakai gz_sim.launch.py dari package ros_gz_sim.
PythonLaunchDescriptionSource menandai bahwa file tersebut berupa Python. Kita memanfaatkan pembuka Gazebo yang sudah tersedia.
launch_arguments mengirim pengaturan ke launch itu. gz_args berisi argumen simulator; -r membuat simulasi langsung berjalan. PathJoinSubstitution menyusun lokasi worlds/diff_drive.sdf, yaitu arena yang akan memuat robot.

```
pkg_project_gazebo = get_package_share_directory(
    'ros_gz_example_gazebo')
```

get_package_share_directory() mencari folder aset package yang sudah di-install. Karena itu, lokasi file tidak perlu ditulis sebagai alamat laptop tertentu. Jika kamu mengedit source, build ulang lalu source lingkungan agar hasil terbaru yang dipakai.
World mengaktifkan sistem Physics, UserCommands, SceneBroadcaster, dan Sensors. Masing-masing menangani fisika, perintah perubahan entity, tampilan scene, dan pengolahan sensor. Model robot sendiri berada di models/diff_drive/model.sdf.

Gazebo Simulation & RViz2

Blok model robot dan RViz2

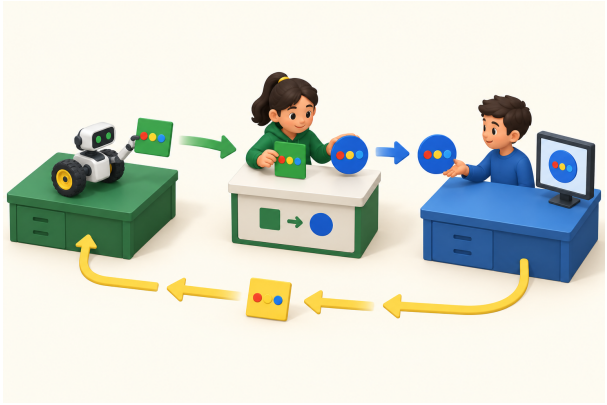
```
robot_state_publisher = Node(
    package='robot_state_publisher',
    executable='robot_state_publisher',
    name='robot_state_publisher',
    output='both',
    parameters=[
        {'use_sim_time': True},
        {'robot_description': robot_desc},
    ]
)
```

package memilih package asal program, executable memilih programnya, dan parameters memasang pengaturan node. robot_desc berisi teks visual.urdf. robot_state_publisher memakai URDF dan /joint_states untuk menerbitkan hubungan posisi antarlink (TF). Gerak fisik roda tetap ditangani Gazebo.

```
rviz = Node(
    package='rviz2',
    executable='rviz2',
    name='rviz2',
    arguments=['-d', os.path.join(
        pkg_project_bringup, 'config', 'diff_drive.rviz')],
    parameters=[{'use_sim_time': True}],
    condition=IfCondition(LaunchConfiguration('rviz'))
)
```

RViz2 memakai -d untuk membaca config/diff_drive.rviz: daftar display, topic, kamera, dan Fixed Frame. arguments adalah argumen command line, sedangkan parameters adalah pengaturan ROS. use_sim_time=True membuat node mengikuti /clock dari simulasi.
IfCondition membaca argumen launch rviz. Tambahkan rviz:=false pada perintah launch jika ingin menjalankan simulator tanpa jendela RViz2.

Bridge: penerjemah pesan



Gambar 7. Perumpamaan penerjemah: informasi tetap dikenali, tetapi bentuk pesannya disesuaikan untuk pihak yang menerima.

Node parameter_bridge dari ros_gz_bridge membaca config/ros_gz_example_bridge.yaml. Bayangin dua orang memakai format alamat berbeda: bridge mengubah formatnya agar pesan bisa diteruskan. Sensor baru perlu entri bridge sendiri agar datanya muncul di ROS 2.

```
- ros_topic_name: /cmd_vel
  gz_topic_name: /cmd_vel
  ros_type_name: geometry_msgs/msg/Twist
  gz_type_name: gz.msgs.Twist
  direction: ROS_TO_GZ
```

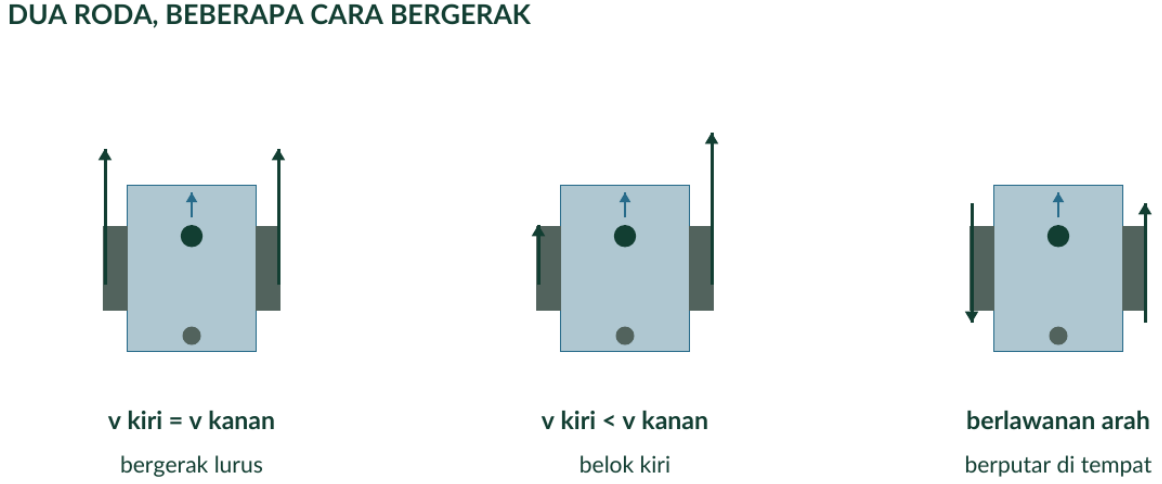
ros_topic_name dan gz_topic_name memilih saluran pada masing-masing sisi; ros_type_name dan gz_type_name memilih format pesannya. ROS_TO_GZ berarti perintah dari ROS 2 menuju Gazebo. Nama topic boleh berbeda, asalkan pasangan dan tipenya benar.

```
- ros_topic_name: /scan
  gz_topic_name: /scan
  ros_type_name: sensor_msgs/msg/LaserScan
  gz_type_name: gz.msgs.LaserScan
  direction: GZ_TO_ROS
  qos_profile: SENSOR_DATA
```

Data sensor mengalir sebaliknya: GZ_TO_ROS. qos_profile: SENSOR_DATA memakai profil komunikasi sensor. YAML memakai indentasi untuk menunjukkan struktur; gunakan spasi dan ikuti susunan file aslinya.

Gazebo Simulation & RViz2

DiffDrive: perintah menjadi gerakan



Gambar 8. Dilihat dari atas: kecepatan kedua roda sama menghasilkan gerak lurus; perbedaan kecepatan membuat robot berbelok.

```
<plugin filename="gz-sim-diff-drive-system"
        name="gz::sim::systems::DiffDrive">
  <left_joint>left_wheel_joint</left_joint>
  <right_joint>right_wheel_joint</right_joint>
  <wheel_separation>1.25</wheel_separation>
  <wheel_radius>0.3</wheel_radius>
  <topic>/cmd_vel</topic>
  <odom_topic>/odom</odom_topic>
  <!-- pengaturan lain ada di file lengkap -->
</plugin>
```

left_joint dan right_joint memilih joint roda yang dikendalikan. wheel_separation adalah jarak antarroda, sedangkan wheel_radius adalah jari-jari roda. Keduanya dipakai untuk menghubungkan kecepatan badan robot dengan putaran roda.

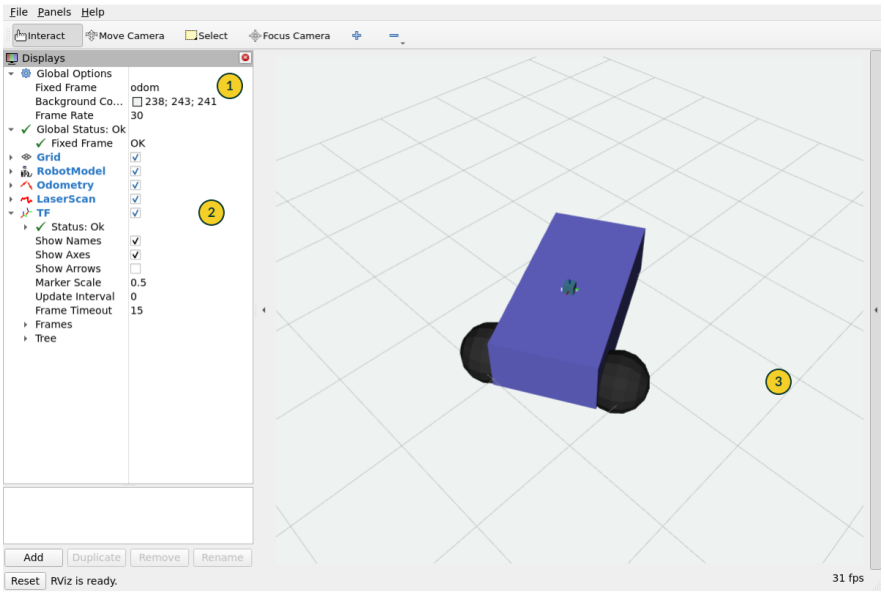
Kalau jarak roda b, kecepatan maju v, dan kecepatan putar ω, kecepatan linear roda kirinya v − ωb/2 dan roda kanannya v + ωb/2. Masing-masing dibagi jari-jari roda untuk mendapat kecepatan sudut roda.

Tidak perlu menghitungnya saat memakai teleop. Yang penting, konfigurasi ukuran roda harus cocok dengan model. Angka yang salah bisa membuat gerak dan odometri tidak sesuai yang kamu harapkan.

Gazebo Simulation & RViz2

Melihat robot di RViz2

RViz2 · model dan hubungan frame



1 Fixed Frame menggunakan odom.
2 TF aktif dan Status: Ok.
3 RobotModel digambar dari model pendamping.

Gambar 9. Tampilan model robot dan display TF pada RViz2. Status Ok membantu memastikan data dan hubungan frame sudah bisa dipakai.

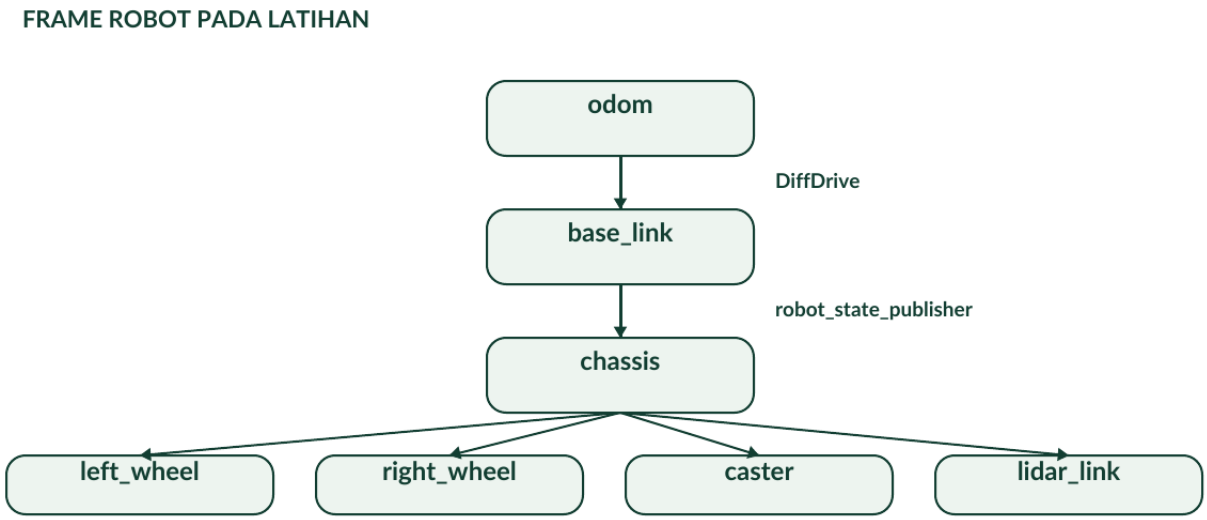
Pilih Fixed Frame = odom. Ini acuan tempat semua data digambar. RobotModel menampilkan bentuk dari /robot_description, TF menampilkan sumbu frame, Odometry membaca /odom, dan LaserScan membaca /scan. Odometri adalah perkiraan perjalanan robot dari geraknya. Pesan /odom memuat pose (posisi dan orientasi) serta twist (kecepatan). Odometri bukan peta dan tidak selalu sama dengan posisi sebenarnya; selisih bisa bertambah selama robot bergerak.

Coba maju dan berputar pelan. Bandingkan badan robot di Gazebo dengan model dan sumbu di RViz2. Jika RobotModel kosong, periksa Description Topic /robot_description dan durability Transient Local supaya deskripsi terakhir bisa diterima.

```
ros2 topic echo /odom --once
```

Gazebo Simulation & RViz2

Frame dan TF: dilihat dari mana?



Gambar 10. Rantai frame pada model pendamping. Transform odom ke base_link berasal dari DiffDrive; hubungan link lain dihitung robot_state_publisher.

Frame adalah acuan koordinat. Bayangin kamu bilang “baloknya satu meter di depan”: di depan robot atau di depan ruangan? base_link mengikuti badan robot, lidar_link mengikuti sensor, dan odom menjadi acuan perjalanan.

TF menyimpan hubungan posisi dan orientasi antarframe. Pada robot ini, DiffDrive memberi hubungan odom ke base_link. robot_state_publisher melanjutkannya ke chassis, sensor, dan roda menggunakan URDF serta /joint_states.

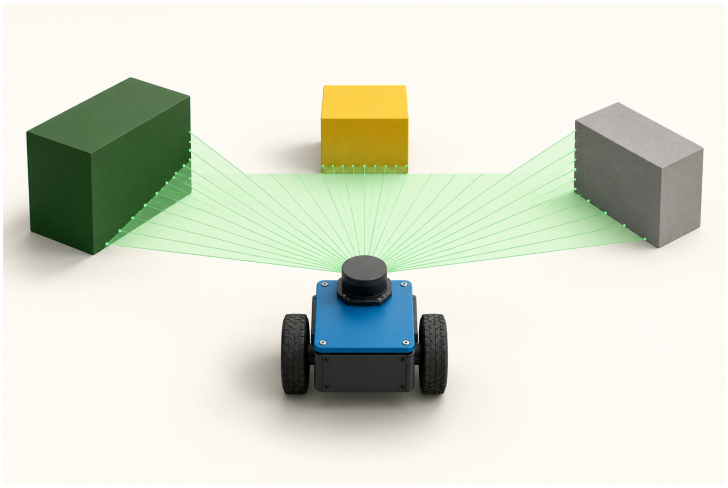
```
ros2 run tf2_ros tf2_echo odom lidar_link
```

Jalankan di terminal yang sudah di-source. Translation dan rotation menunjukkan pose lidar_link terhadap odom. Gerakkan robot untuk melihat perubahan, lalu Ctrl+C untuk berhenti membaca.

Kalau jalur lidar_link ke odom putus, RViz2 bisa menerima /scan tetapi tidak tahu harus menggambarnya di mana. Jadi, pesan masuk saja belum cukup: frame-nya juga harus terhubung.

Gazebo Simulation & RViz2

LiDAR dan blok sensornya



Gambar 11. Perumpamaan LiDAR planar: tiap arah pengukuran memberi jarak ke permukaan terdekat yang terdeteksi.

LiDAR mengukur jarak pada banyak arah. Bayangin senter yang menyapu ke kiri dan kanan: setiap arah menghasilkan satu jarak ke permukaan. Sensor simulasi menghitung perpotongan sinar dengan objek; ini bukan foto atau peta lengkap.

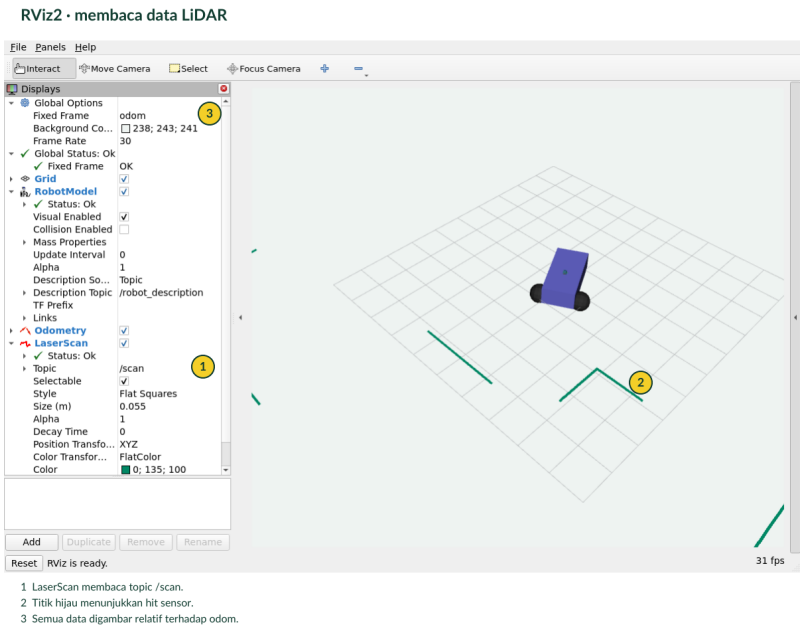
```
<sensor name="gpu_lidar" type="gpu_lidar">
  <topic>scan</topic>
  <gz_frame_id>lidar_link</gz_frame_id>
  <update_rate>10</update_rate>
  <lidar>
    <!-- scan horizontal dan range ada di file lengkap -->
  </lidar>
</sensor>
```

Buka model.sdf untuk melihat blok lengkap. topic menentukan keluaran Gazebo, gz_frame_id memberi acuan sensor, dan update_rate=10 meminta 10 pengukuran per detik waktu simulasi. Sistem Sensors pada world juga harus aktif.

Pada bagian scan, samples=640 berarti 640 sampel horizontal. min_angle dan max_angle membatasi sudut, sekitar −80° sampai +80°. Bagian range membatasi jarak 0,08–10 meter. Satuan sudut di kode adalah radian.

Gazebo Simulation & RViz2

Membaca hasil scan



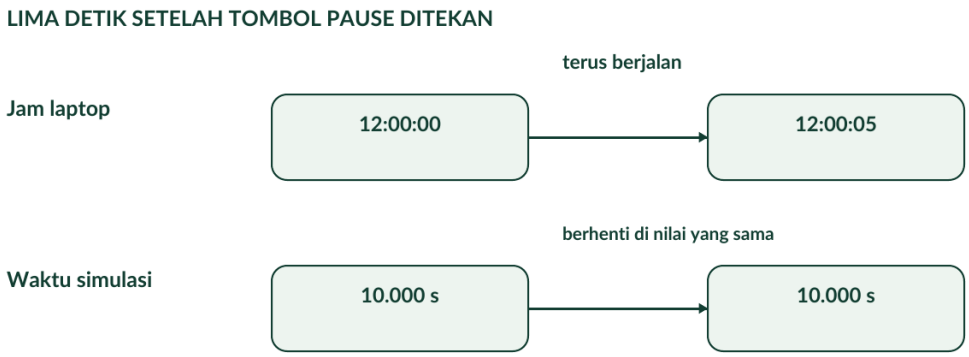
Gambar 12. Titik hijau pada RViz2 adalah hasil pembacaan /scan. Bagian tepi yang terlihat lurus bisa berasal dari sisi balok atau batas arena.

```
ros2 topic echo /scan --once --qos-reliability best_effort
```

Di RViz2, pilih topic /scan pada display LaserScan dan Reliability = Best Effort. Titik-titik seharusnya mengikuti permukaan rintangan yang terlihat sensor. Sisi belakang balok tidak otomatis muncul karena terhalang sisi depannya.
Dalam pesan LaserScan, ranges adalah deretan jarak dalam meter. angle_min adalah arah sampel pertama, angle_increment adalah selisih sudut antarsampel. Sampel ke-i memiliki sudut angle_min + i × angle_increment, dengan indeks mulai dari nol.
header.frame_id memberi nama acuan dan header.stamp menandai waktu pengukuran. range_min/range_max membatasi jarak yang valid. Nilai inf pada simulasi ini berarti tidak ada hit valid dalam jangkauan; jangan menganggapnya jarak rintangan biasa.

Gazebo Simulation & RViz2

Waktu simulasi dan QoS



Gambar 13. Ketika simulasi dijeda, jam laptop tetap berjalan. Nilai waktu simulasi tidak maju sampai world dilanjutkan.

Jam laptop terus berjalan ketika Gazebo di-pause, tetapi waktu simulasi berhenti maju. Sensor dan TF harus memakai acuan waktu yang selaras agar RViz2 bisa memasangkan data dengan posisi robot pada saat yang tepat.

```
ros2 topic echo /clock --once
ros2 param get /rviz2 use_sim_time
ros2 param get /robot_state_publisher use_sim_time
```

use_sim_time pada RViz2 dan robot_state_publisher seharusnya True. Timestamp adalah penanda waktu data; jika sebagian node memakai jam laptop, data bisa dianggap terlalu lama atau terlalu baru.

Kenapa scan memakai Best Effort?

QoS (Quality of Service) mengatur cara pengiriman pesan. Reliable memakai mekanisme untuk membantu pesan sampai. Best Effort membolehkan sebagian pesan terlewat; profil sensor memakainya untuk aliran data yang terus diperbarui.

Subscriber Reliable tidak cocok dengan publisher yang hanya menawarkan Best Effort. Karena itu, samakan LaserScan RViz2 dengan bridge /scan yang memakai Best Effort. Durability adalah pengaturan QoS lain: Transient Local membantu pembaca baru menerima data terakhir yang disimpan publisher.

Coba geser satu rintangan

Hentikan teleop dan launch sebelum mengubah world. Buka ros_gz_example_gazebo/worlds/diff_drive.sdf dari folder source, lalu cari model yellow_obstacle.

```
<model name="yellow_obstacle">
  <static>true</static>
  <pose>6 0 0.6 0 0 0</pose>
  <!-- bagian link, collision, dan visual tetap -->
</model>
```

Nilai awal x-nya 5. Pada contoh perubahan ini, kita ubah menjadi 6 supaya balok bergeser satu meter pada sumbu x world. Nilai z tetap 0,6 agar bagian bawah kotak tetap berada di lantai.

```
bash scripts/build.sh
source scripts/setup_env.bash
ros2 launch ros_gz_example_bringup diff_drive.launch.py
```

Muat ulang simulasi agar file world dibaca lagi. Sekarang bandingkan jarak scan pada arah yang menghadap balok tersebut. Kalau posisi robot sama dan garis pandangnya tetap mengenai balok, titik permukaannya seharusnya terlihat lebih jauh.
Setelah mencoba posisi, kamu bisa mencoba ukuran atau warna. Ubah satu hal setiap percobaan. Dengan begitu, ketika hasilnya berbeda, kamu tahu perubahan mana yang paling mungkin menjadi penyebabnya.
Ingat urutan pose: x y z roll pitch yaw. static=true membuat rintangan tetap diam. Jika nanti mengubah ukuran, sesuaikan visual dan collision agar bentuk yang terlihat cocok dengan bentuk tabrakannya.

Kalau hasilnya belum sesuai

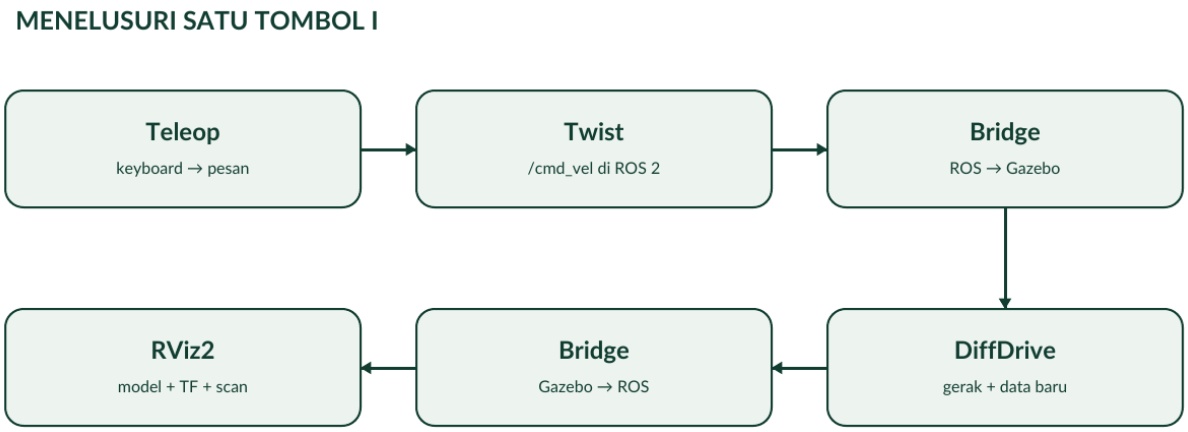
Yang kamu lihat	Yang bisa dicoba
Package tidak ditemukan	Source scripts/setup_env.bash dari folder modul. Pastikan build selesai dan terminal tidak memuat distro ROS lain.
World atau model gagal dimuat	Baca lokasi file pada Terminal A. Build dari source yang dibagikan, lalu source ulang; periksa nama file dan URI model.
Robot tidak bergerak	Lanjutkan simulator dari pause, fokuskan keyboard ke teleop, lalu periksa /cmd_vel serta bridge Harmonic.
Model tidak terlihat di RViz2	Pilih Fixed Frame=odom. Periksa /robot_description dan TF; gunakan Transient Local pada Description Topic.
LaserScan kosong atau error	Pilih /scan dan Best Effort. Periksa frame lidar_link, jalur TF, dan waktu simulasi.
Timestamp tidak cocok	Periksa use_sim_time pada RViz2 dan robot_state_publisher serta aliran /clock.
Jendela berat atau rendering gagal	Tutup aplikasi grafis berat. Periksa dukungan OpenGL; software rendering di bawah bisa dicoba, tetapi dapat lebih lambat.

```
export LIBGL_ALWAYS_SOFTWARE=1
ros2 launch ros_gz_example_bringup diff_drive.launch.py
```

Pesan Stereo is NOT SUPPORTED pada RViz2 tidak otomatis berarti tampilan biasa gagal. Cocokkan pesan terminal dengan keadaan display dan data yang benar-benar terlihat.

Gazebo Simulation & RViz2

Coba telusuri datanya



Gambar 14. Satu tombol melibatkan beberapa bagian. Nama topic dan konfigurasi yang cocok membuat rantai ini tersambung.

Buka Terminal C dari folder modul. Biarkan simulasi dan teleop tetap berjalan. Perintah berikut membantu melihat bagian mana yang sudah bekerja:

```
source scripts/setup_env.bash
ros2 topic list
ros2 topic info /cmd_vel
ros2 topic echo /cmd_vel
```

Tekan i lalu k di Terminal B. Nilai linear.x pada Terminal C seharusnya berubah, lalu kembali nol. topic info menunjukkan tipe pesan serta jumlah publisher/subscriber; echo menampilkan isi pesan. Ctrl+C hanya menghentikan pembacaan echo.

Sekarang coba berputar dengan j. Amati angular.z, arah badan robot, dan perubahan scan. Jika /cmd_vel berubah tetapi robot diam, periksa pause, bridge, dan DiffDrive. Jika robot bergerak tetapi scan kosong, periksa sensor, bridge /scan, QoS, lalu TF.

Untuk percobaan berikutnya, geser satu balok seperti halaman sebelumnya dan bandingkan scan sebelum/sesudah. Ubah satu hal setiap percobaan agar hubungan antara perubahan kode dan hasilnya mudah terlihat.

Source dan bacaan lanjutan

Package latihan diadaptasi dari ros_gz_project_template milik organisasi Gazebo, dengan revisi 045e981d03aac1cc3712ab6bb086f52ea79aebf9. Nama package dan lisensi Apache-2.0 dipertahankan. Daftar perubahan ada di UPSTREAM.md.

Template resmi Gazebo
Panduan struktur template
Instalasi Harmonic pada Ubuntu
Integrasi ROS 2 dan Gazebo
Referensi sistem DiffDrive
Panduan penggunaan RViz2
Penjelasan QoS ROS 2

Copyright template: Open Source Robotics Foundation, Inc. Penulis package mencakup Michael Carroll dan Dharini Dutia. Adaptasi pembelajaran ini bukan rilis resmi upstream.

Sampul dan latar halaman berasal dari aset RIVAL yang disediakan. Ilustrasi perumpamaan dibuat dengan imagegen, sedangkan diagram teknis digambar dari kode agar label dan panahnya terkontrol. Screenshot Gazebo dan RViz2 diambil dari aplikasi yang benar-benar berjalan.